AZURE MEETUP PUGLIA

BY COMMUNITY · FOR COMMUNITY

#GLOBALAZURE

2026

BENVENUTI

GLOBAL AZURE PUGLIA 2026

Venerdì 17 aprile - Bari

Grazie Agli Sponsor

SPECTRAL COMPUTE | SCALE

Ahead-Of-Time Compilation of CUDA for Microsoft Azure VMs with AMD GPUs

That's a mouthful. What about "spend less on Azure for better performance"?

tl;dr

- **MS Azure is investing in AMD GPUs**

  Azure now offers **AMD MI300X** and **Radeon Pro V710** VMs.

- **Cheaper and better-specced**

  e.g. AMD MI300X has **240% the amount of VRAM** of NVIDIA H100, **but costs ½**.

- **Achieve vendor optionality on MS Azure**

  SCALE compiles for both NVIDIA and AMD targets. One codebase, better compiler tooling, increased performance.

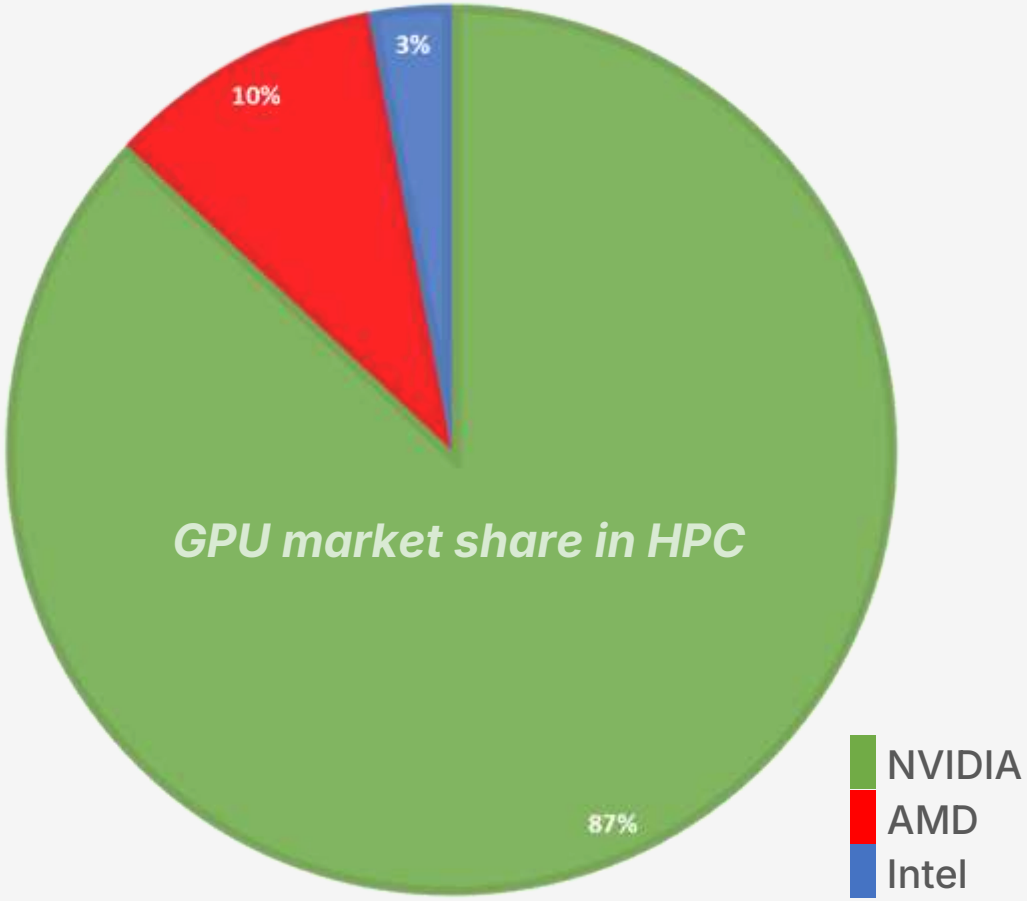
SCALE

the Azure dev community!

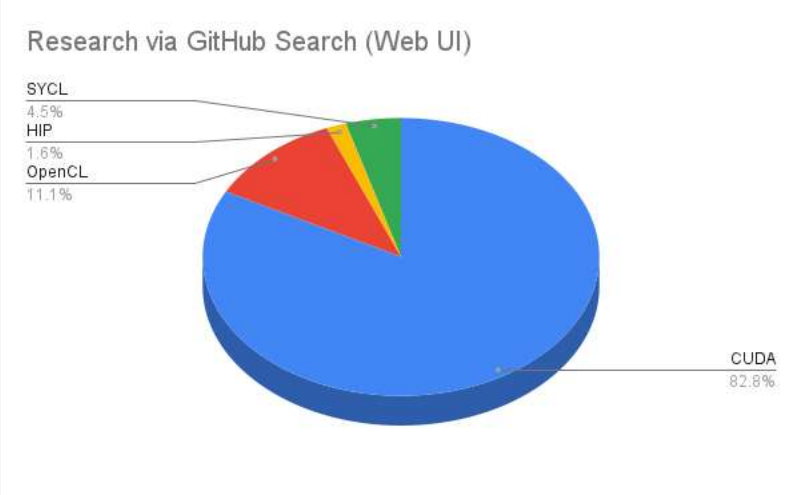
Challenge: Vendor lock-in is slowing down the HPC industry

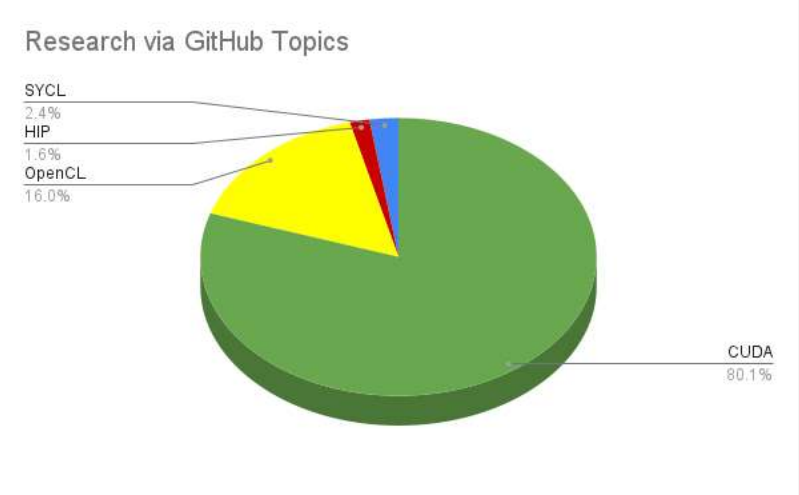
CUDA is *the* most widely-used accelerator programming language.

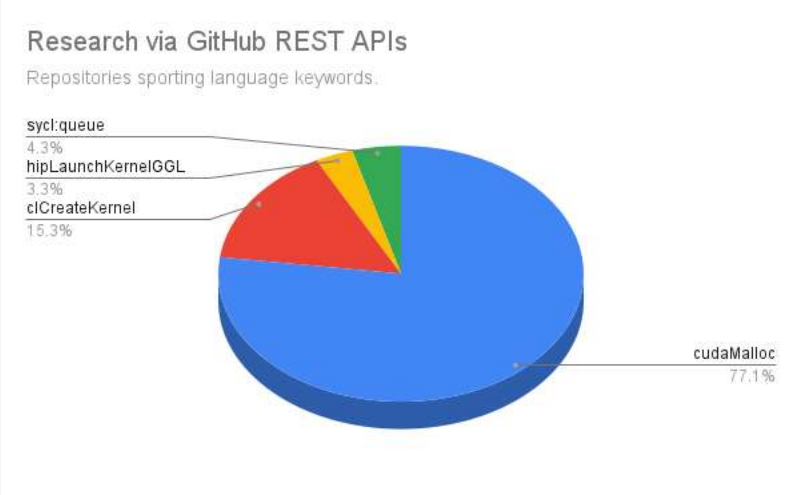
- → Huge ecosystem of libraries, tools, frameworks etc.
- → 5+ million registered developers
- → 20 years of development, numerous OSS CUDA projects
- → Broad adoption across major industries and research
- → CUDA **ONLY** supports Nvidia GPUs

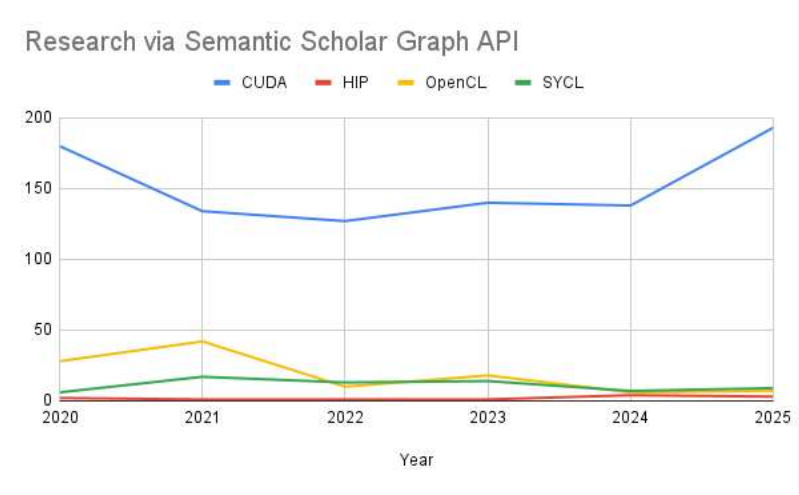


CUDA: The De-Facto Standard of High Performance Computing









Source: [CUDA, the De Facto Standard of HPC? | SCALE GPGPU Programming Language]

Price&spec comparison: What the CUDA lock-in means in $

Source: cloudprice.net, calculated WRT East US (2) (lowest price-points), 13 April 2026.

Tier	Azure SKU	GPU	Per-VM memory	On-demand $/hr
AI/HPC flagship	ND MI300X v5 series (ND96isr MI300X v5)	8× AMD Instinct MI300X	1536 GB HBM3	**$48.00**
AI/HPC flagship	ND H100 v5 series (ND96isr H100 v5)	8× NVIDIA H100	640 GB HBM3	**$98.32**

A CUDA codebase is an implicit hardware contract

→ C/C++/Rust can be compiled for ARM/Intel/AMD CPUs.

*Why can't **CUDA** be like that?!*

Existing solutions emphasise *porting away* from CUDA

- **hipify/SYCLomatic:** semi-automated porting
- **OpenCL/triton/Mojo etc.:** alternative GPU languages

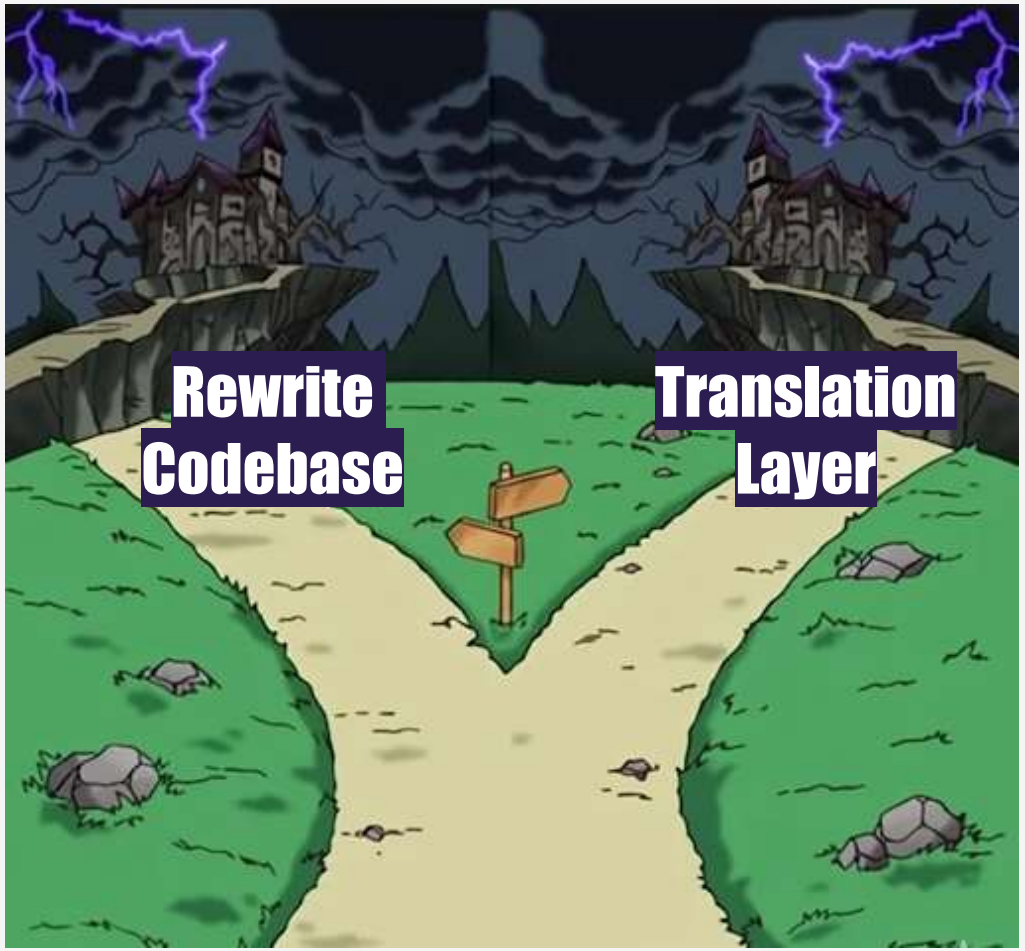
Portability through rewriting is not portability. It's rewriting.

- Maintainers frequently keep multiple versions of codebases
- Engineers forced to pick tools based on vendor compatibility

The Switching Nightmare: A Choice Between Two Bad Options

- Until now, your choice was stark: want to run your CUDA code on non-NVIDIA hardware?

  Either undertake a costly, multi-year project or accept the risks and performance gaps of a translation layer.



GPU portability: A maze of dead ends

ZLUDA	HIP	OpenCL	Mojo
+ "CUDA Compatible" + Can support (most) programs without recompilation	+ Open source + "Looks like CUDA"	+ Open standard + Supported by most vendors	+ Supports NVIDIA and AMD + Cool new language
- Requires use of NVIDIA toolkits - Being rewritten because of legal clawback from AMD - No AMD data-center grade GPU support - Cannot match performance of native CUDA - Uses HIP runtime - Against the nvcc EULA	- Requires significant effort in porting and subsequent maintenance - Mainly targets AMD (NVIDIA through largely untested backend) - Significantly smaller ecosystem - Significant stability and support issues[1]	- Restricted language features - Significantly smaller ecosystem	- Significant porting and migration efforts from CUDA required - In making "Python" performant, it ends up looking like CUDA/C++ without getting the ecosystem benefits

[1]: MI300X vs H100 vs H200 Benchmark Part 1: Training – CUDA Moat Still Alive – SemiAnalysis

Issues with automatic source translation

Tools like hipify partially automate porting away from CUDA.

Challenges:

- Inline asm() is common in CUDA code and is missed by source translation
- Dialect problem: NVIDIA CUDA and Clang-CUDA are subtly different languages;
- CUDA API calls are often assembled by macros;
- HIP's APIs behave subtly differently from CUDA's.

↪ Finished? Now you have two codebases to maintain 😭😭😭

The “CUDA Dialect Problem”

The “standard” is defined by NVIDIA’s nvcc behavior.
As the LLVM documentation puts it:

Dialect Differences Between clang and nvcc

There is no formal CUDA spec, and clang and nvcc speak slightly different dialects of the language. Below, we describe some of the differences.

This section is painful; hopefully you can skip this section and live your life blissfully unaware.

This section is painful;

→ *What could possibly go wrong?*

ZLUDA: Or, why other approaches fail

Security risk

ZLUDA's reliance on DLL injection is frequently flagged as malware-like behavior by Windows Defender, making it non-viable for enterprise deployment.

Theoretically flawed

It suffers from the unfixable "decompiler problem". It starts from code already compiled for NVIDIA, losing critical information needed for re-optimization on AMD architectures.

SCALE avoids these issues entirely by **working from the original CUDA source code**, ensuring a robust, secure, and performant solution.

SCALE: A CUDA Superset and Cross-Compilation Toolkit

→ Our approach: **Write CUDA code, Run Everywhere**

Target other GPU vendors

Drop-in replacement for the Nvidia **nvcc** compiler. Same command line, same semantics, but with NVIDA and *non-NVIDIA GPU support*.

CUDA Runtime and Driver API

A new cross-platform implementation of the CUDA Driver/Runtime APIs, abstracting over the underlying hardware.

CUDA Math API

Our own implementation of the CUDA Math API and other device-side APIs, built for any platform that SCALE supports.

"CUDA-X" Math libraries

Rewrite of closed source libraries such as cuBLAS in CUDA, or wrap vendor-provided options like AMD's rocBLAS.

Build and test FOSS CUDA projects

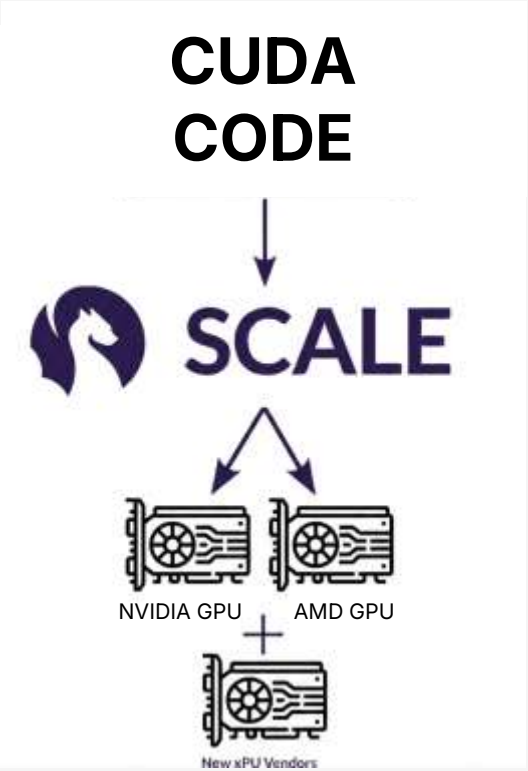
SCALE validated by building and testing "real-world" CUDA code. Nothing beats complex GitHub projects when it comes to finding problems.

SCALE: Beyond CUDA

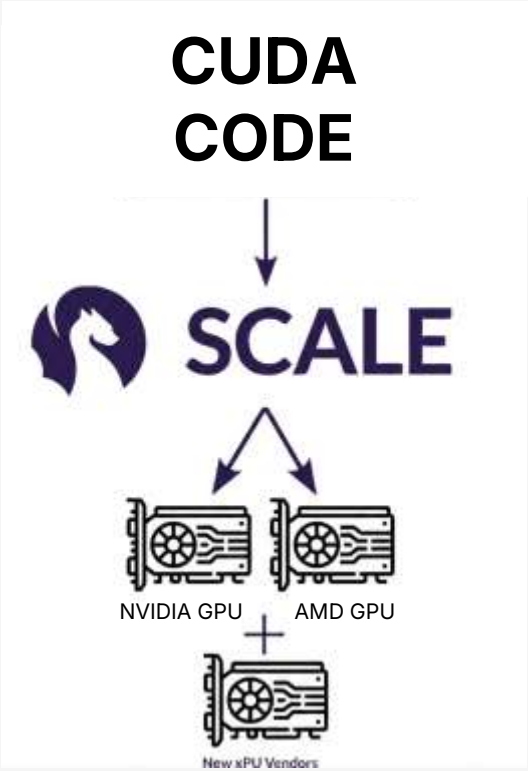
What is SCALE?

SCALE brings the paradigm of CPU programming over to HPC computing platforms: write once, recompile for different hardware.

- It's a toolkit: an **nvcc** drop-in replacement compiler, CUDA Language Extensions, open source libraries providing the "CUDA-X" APIs (e.g. cuBLAS, cuSOLVER, etc).
- It compiles a superset of CUDA to AMD machine instructions.
- Not an emulator. No intrinsic overhead.



SCALE: Beyond CUDA



Why SCALE?

- By supporting CUDA natively: avoid maintaining multiple versions of code for different vendors.
- Select hardware based on its properties, not vendor-lock-in. Example: AMD hardware is better at high-precision workloads
- SCALE complements accelerator hardware and will lead to beating this vendor-lock.
- Hardware vendors are not incentivized to optimize performance - they want to sell HW, and the newest HW has the best performance!

CUDA Coverage

SCALE currently tests:

- 23 open-source CUDA projects
- 14 AMD GPU architectures
- 27 NVIDIA GPU architectures

To run all of those, SCALE covers:

- ~90% of the CUDA 13 runtime API
- 70% of the CUDA driver API
- ~95% of the CUDA math API
- ~75% cuSOLVER/cuSPARSE
- 100% cuBLAS, cuFFT, cuRAND

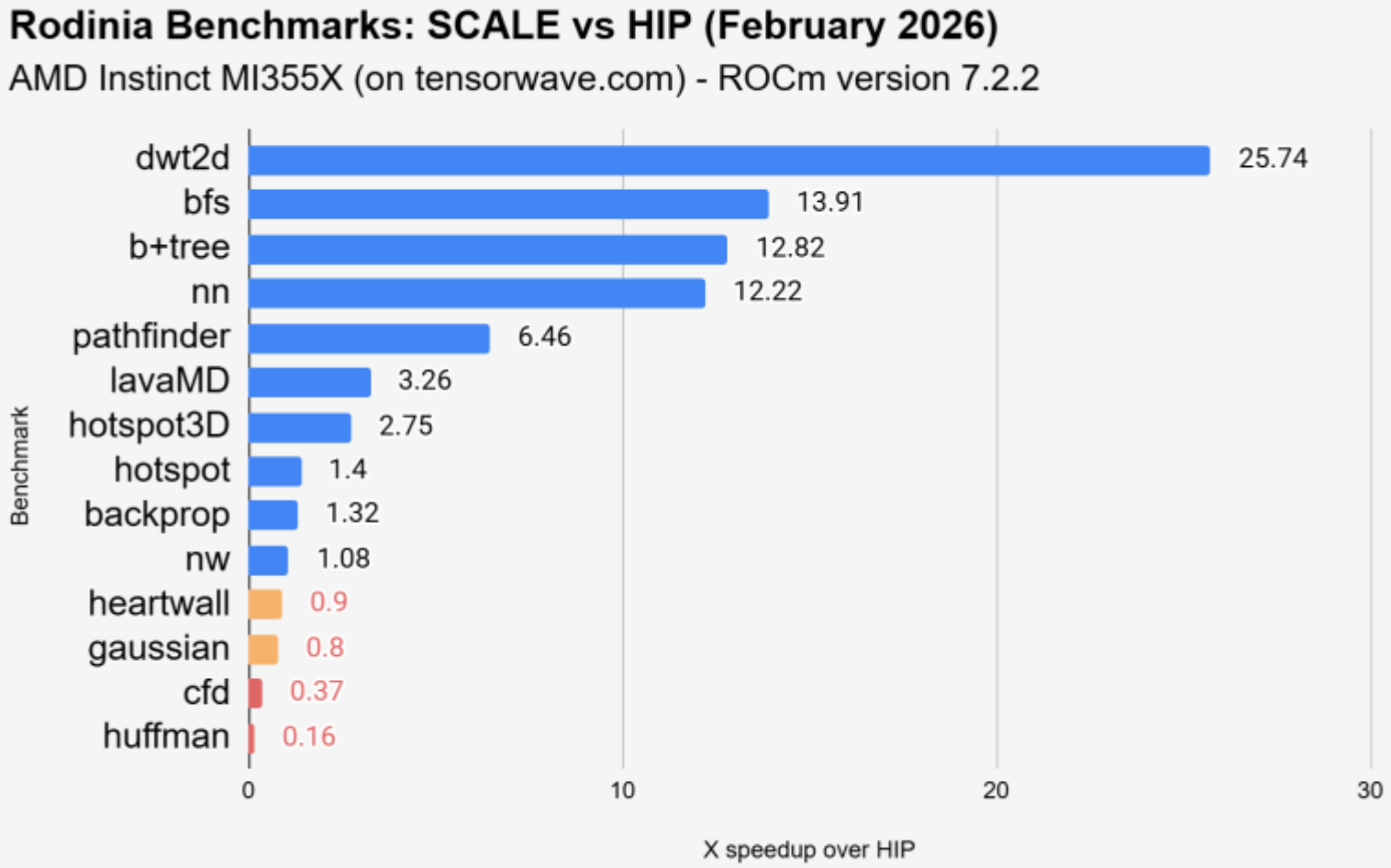
Run on **AMD only with SCALE:**

Project	SCALE	HIP
amgX	✓	✗
FLAMEGPU2	✓	✗
Faiss	✓	✗
alien	✓	✗
XGBoost	✓	✗
GOMC	✓	✗
gpujpeg2k	✓	✗
CUTLASS	✓	✗

Multiple code bases: HIP for AMD and CUDA for NVIDIA

Project	SCALE	HIP
Hashcat	✓	✓
GROMACS	✓	✓
cycles	✓	✓
Llama.cpp	✓	✓
GPUJPEG	✓	✓
stdGPU	✓	✓
Thrust	✓	✓

Performance: SCALE on AMD

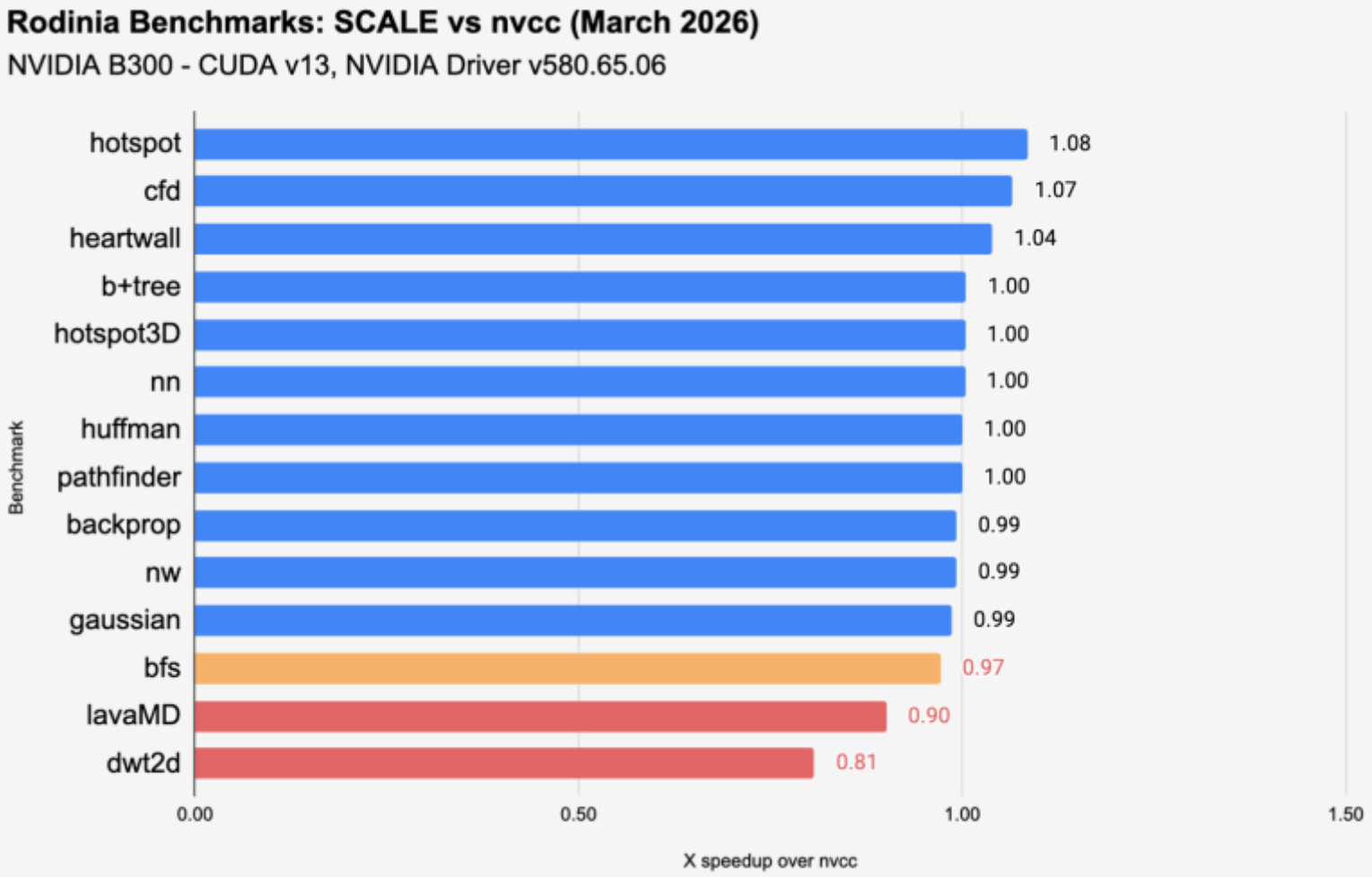


Average speedup over HIP:

5.94x

- Blue faster than HIP
- Orange close to parity
- Red currently slower

Performance: SCALE on NVIDIA



Up to

~8% faster

than nvcc

- Blue faster than nvcc
- Orange close to parity
- Red currently slower

***NOTE**: SCALE Nvidia support released in Mar 2026. Novel SCALE compiler IP **not** yet activated. Near term performance gains expected.*

The cherry on the cake: Superior CUDA Developer Experience

LSP Server

Semantic code intel

Clear Diagnostics

Human readable errors

Agent Readiness

Consumes LSP + errors

```
17    asm("add.f32 %0, %1, %2;" : "=l"(u) : "r"(u), "f"(f));  // expected-error {{PTX constraint '=l' requir
18    asm(
19    asm(
20  }
21
22  // Att
23  __glob
24    int
25    int
26    int
27    asm("addc.u32 %0, %1, %2;" : "=r"(z) : "r"(x), "r"(y));
28    createCarryBit(x, y);
29    asm("addc.cc.u32 %0, %1, %2;" : "=r"(z) : "r"(x), "r"(y));
30  }
```

Instruction reads undefined value for PTX carry-bit. The carry-bit is not preserved across C++ function boundaries on any platform, so it must be written to within the same function before it be read safely clang(ptx_undefined_carry_in)

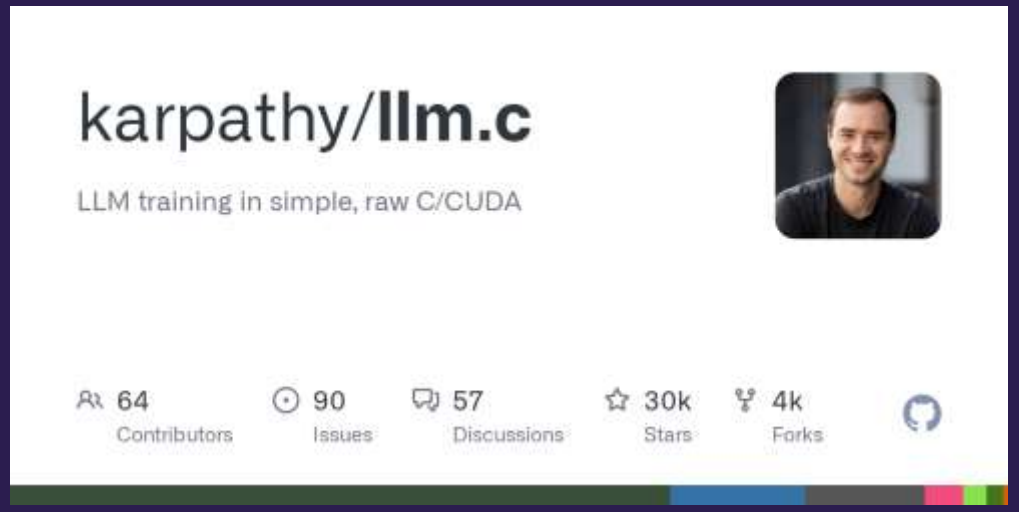
string-literal

Type: const char[21]
Size: 21 bytes

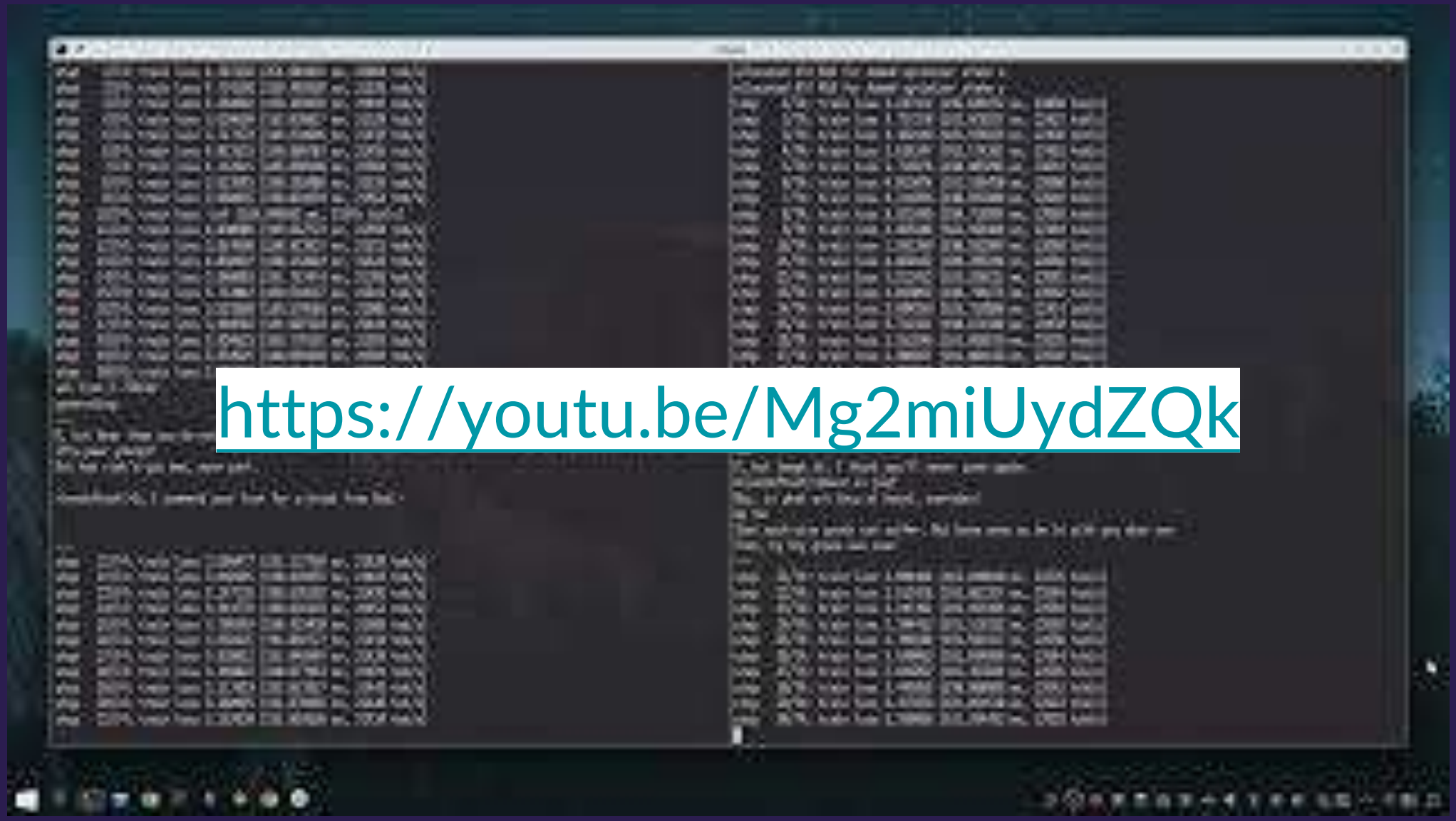
View Problem (Alt+F8) No quick fixes available

Demo Time

- llm.c via SCALE.



Demo Time



https://youtu.be/Mg2miUydZQk

Demo Time

- ALIEN via SCALE.

Demo Time

SCALE: A CUDA Superset and Cross-Compilation Toolkit

- **True Portability**

  Write CUDA, run everywhere with **zero code rewrite**.

- **Increased Performance**

  Proven speedups across all supported vendors.

- **Single Codebase**

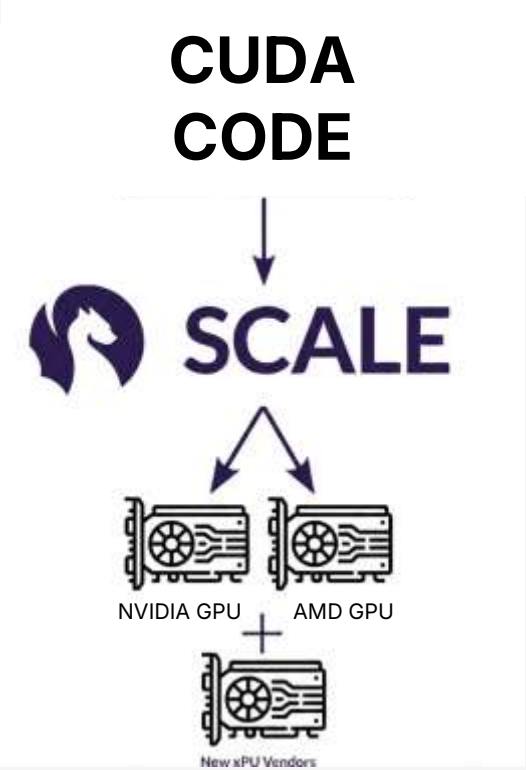
  No overhead of maintaining multiple code forks for different vendors.

- **Better Diagnostics**

  Modern compiler error messaging that **accelerate development**.

- **Hardware Freedom**

  Select VMs on MS Azure based on specs and price, not constraints.



SCALE: Adding value across the chain

Chip Vendors

- **Business Surface**, support for CUDA gives instant access to 80% of *existing* HPC market
- **Augment Ecosystem**, adding CUDA support compounds value of vendors own SDKs

HPC Developers

- **Focus only on CUDA**, master one language instead of many
- **Increase productivity**, access superior tooling, accelerate development
- **One codebase**, smaller bug surface, reduced cost / overhead

Cloud Service Providers

- **Break vendor lock-in**, freedom to choose the best GPU for a workload
- **Reduce CAPEX**, access diverse portfolio of cost-effective GPUs
- **De-risk supply chain**, less vulnerable to unexpected changes

End Users

- **Lower Token Cost**, improved performance drives lower cost per GPU hour
- **Shorter lead time**, CUDA workloads now run on diverse GPU portfolio, increasing availability

Go to Market: Our multi-tier strategy

Non-Commercial

Free

For educational and research purposes (local workstation and data-center GPUs).

e.g. *LUMI / CSC, Tuolumne / LLNL, Frontier / ORNL*

→

Commercial

Paid

Standard license for commercial deployment and use.

Enterprise

Paid

Custom xPU software solutions, optimizations and roadmap prioritization.

→ **Non-Commercial:** Partnerships with research data centers, influencing future generation(s) of scientific computing professionals

→ **Commercial:** Collaboration with Cloud Service Providers, split per-GPU-hour revenue in exchange for CUDA portability and increased performance

→ **Enterprise:** Collaborate with xPU vendors, enabling support for CUDA for the next generation of AI compute accelerators

Traction: SCALE Roadmap 2026+

Q1

Support for Nvidia GPUs, improved performance for existing research supercomputers.

Q2

Support for PyTorch, use SCALE seamlessly in the #1 AI research framework.

Q3

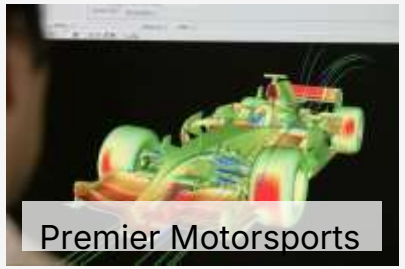
Support for vLLM, SGLang, faster AI model inference with SCALE.

Q4+

Support for xPU, support for new AI accelerators / GPU Vendors

Traction: Adopted Across Industries

→ SCALE by HPC Clients:



→ SCALE on supercomputers:

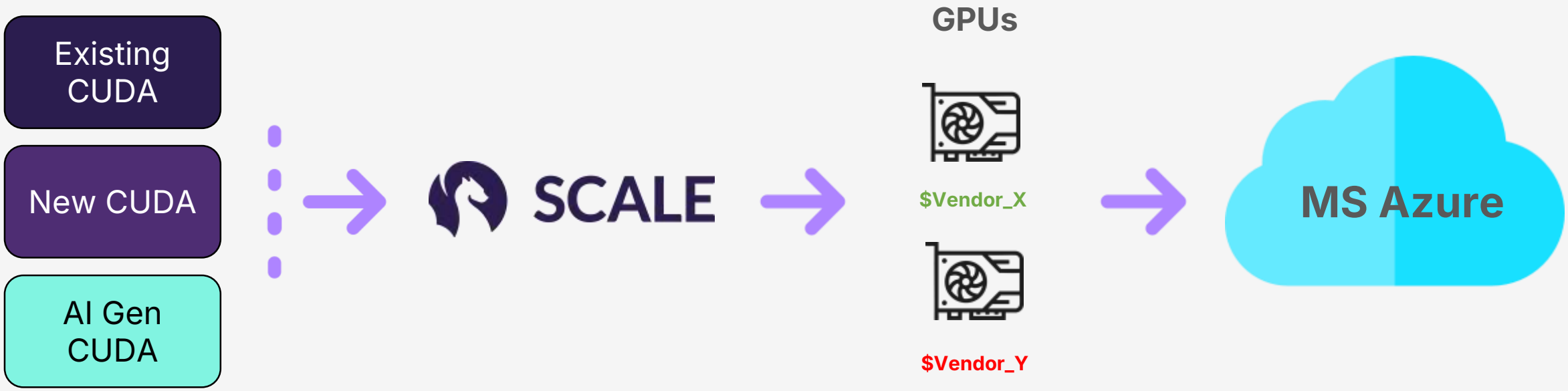
LUMI, Europe's most powerful supercomputer, 375 petaflops.

Tuolumne, unclassified sibling system of El Capitan, 288 petaflops.

SCALE and Azure: A Complementary Strategy

SCALE **augments** current industry, transforming common accelerators into drop-in replacements for the existing market of CUDA workloads.

SCALE makes Azure into a **hybrid vendor cloud**, our take on heterogeneous compute clusters based on the dominant accelerator programming language.



SCALE and Azure: What to bring home

Inference of large open-weight models (Llama-70B, DeepSeek, Mixtral-style MoEs)

- **MI300X's 192 GB per GPU** means **fewer GPUs per replica**, **simpler sharding**, **less interconnect contention**. SCALE lets you keep the CUDA inference kernels you already tuned.

HPC workloads (CFD, MD, finance) with existing CUDA codebases.

- **Typical pattern: a decade of CUDA code no one wants to port.** SCALE compiles it for MI300X; you benchmark; **if it wins on $/FLOP**, **you migrate VM without migrating code**.

VDI and small-model inference on V710.

- SR-IOV partitioning + ROCm for the graphics side, **SCALE for any CUDA-based inference or compute workloads** co-located on the same GPU.

Call to Action

- **Try it.** docs.scale-lang.com — install, compile a CUDA sample, run it on any AMD GPU you have access to (including a V710 VM you can spin up in minutes).
- **Benchmark your workload**. If you have CUDA code and an Azure account (duh?!), the cost of a portability experiment is an afternoon and a few dollars of MI300X or V710 time.
- **Talk to us.** Let's get in touch.

SCALE

↪ Same CUDA codebase, increased performance, better tooling, more $/throughput on Azure.

SPECTRAL COMPUTE | SCALE

SCALE Dev Portal

Discord

SPECTRAL COMPUTE on social media:

Spectral Compute LinkedIn

Spectral Compute Twitter

FEEDBACK